Django Admin or the `ch10h_exercises` sample code directory, which includes a fixture with some data.

One last thing needs to be done before it will work: you need the HTMX JavaScript file. You reference this file in base.html, so it can be loaded into all of your pages. The `<script>` tag that includes the library can reference a CDN, as follows:

```
<script src="https://unpkg.com/htmx.org@1.9.8"></script>
```

> **NOTE** A *content delivery network* (CDN) is a giant cache on the internet, which many popular tools use to centrally locate their content, meaning your browser can cache files common between different sites.

Or if you prefer not to use a CDN, you can download the file for yourself and serve it as static content, like you learned about in chapter 7. This is the approach I've taken, as I don't always trust CDNs to keep their files around for a long time. This comes at the cost of page load time; the user's browser can't use a cached copy from a different site because it doesn't know it is the same content as HTMX elsewhere. The change I made to base.html to load the HTMX library from the project directly is shown in the following listing.

Listing 12.5 Add the HTMX library to the base template

```
<!-- RiffMates/templates/base.html -->
{% load static %}
<head>
  <!-- ... -->
  <script src="{% static 'js/htmx-1.9.8.min.js' %}"
    ></script>

</head>
```

Load the {% static %} tag.

Use the downloaded copy of the library found in RiffMates/static/js/htmx-1.9.8.min.js.

Restart your development server, and visit the `promoters()` view. I put mine at http://localhost:8000/promoters/promoters/. As there is a 2-second delay in the view called by the lazy load, you should see the Loading… message, followed by your list of promoters filled in dynamically.

12.4 Search-as-you-type with infinite scroll

A common dynamic interaction on a web page is search-as-you-type, where a search page has an input box and as you enter search terms, the results are populated below. Another dynamic concept is infinite scroll. Consider your favorite social media feed: as you scroll, new content gets added to the bottom. Both of these concepts can be built with HTMX, requiring little more code than a regular view.

Before considering the HTMX version, let's start with a musicians search view, making sure it works before adding the interactive part. The view uses a query parameter to get the search text—it'll be clear why, when the HTMX part gets wired in.

To search a model, you need to construct a query based on the search text. For a `Musician`, you want to search by first and last names. Instead of looking for exact